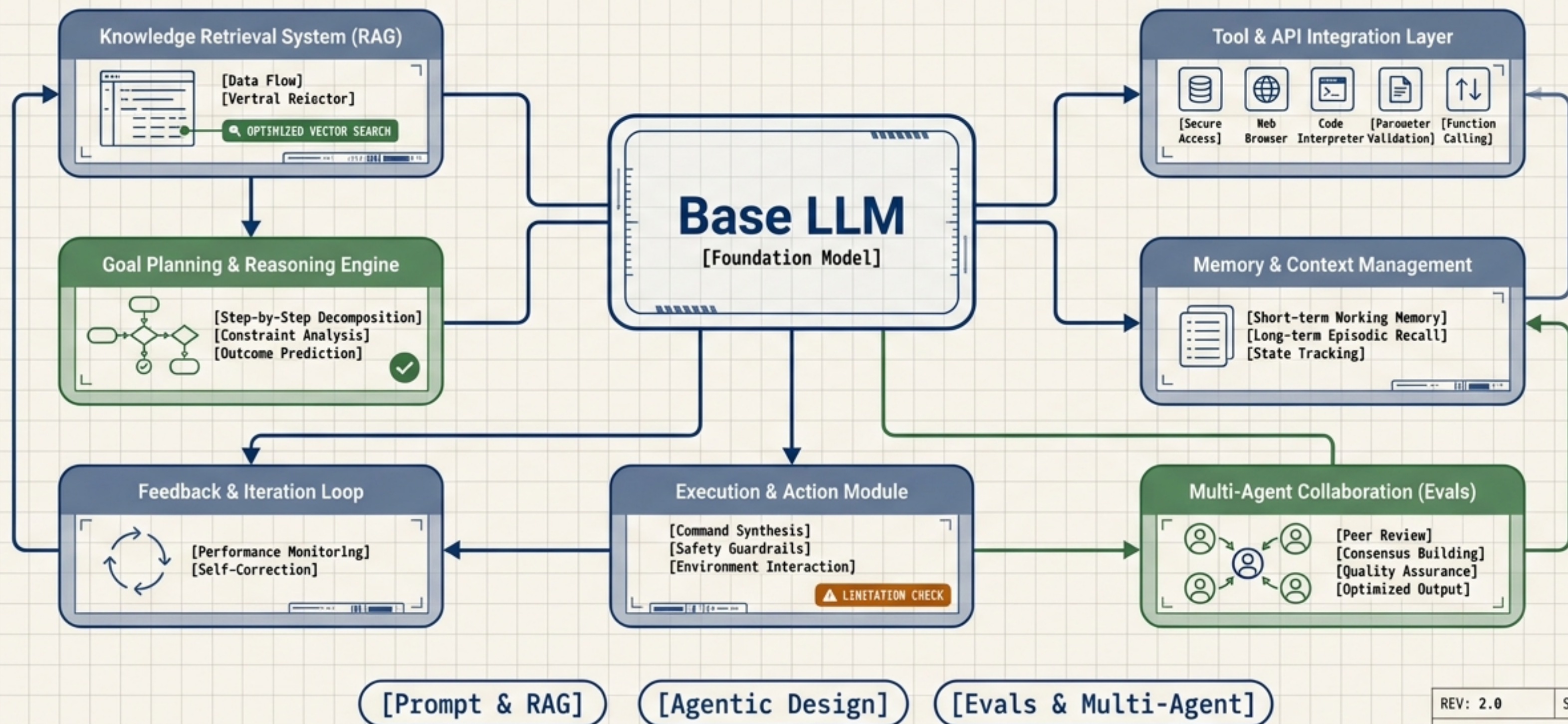


從 LLM 到 Agentic Workflow：AI 系統建構藍圖

史丹佛大學 AI 系統課程精華——跨越「聊天機器人」，打造具備商業價值的自主 AI 系統



REV: 2.0

SCALE: NTS

PROJECT: AGENTIC BLUEPRINT

基礎模型的商業落地撞牆期

LIMITATION

缺乏領域知識 (Domain Knowledge)

模型未曾訓練過企業內部文件或最新產品規格，無法針對封閉數據（如特定農業病害資料）提供解答。



LIMITATION

資訊落後 (Stale Info)

模型無法頻繁重新訓練，無法辨識新詞彙、突發事件或最新網路用語。



LIMITATION

控制困難 (Fuzzy Control)

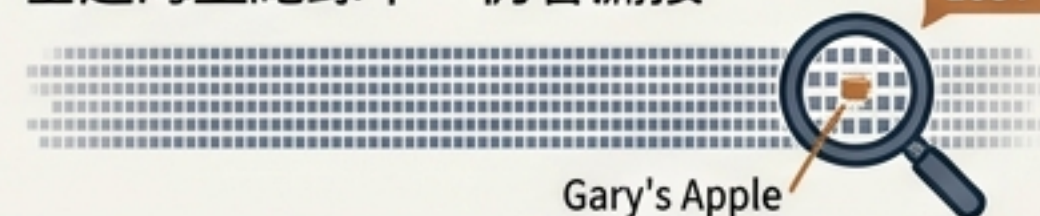
具備機率性隨機輸出特質。在客服等高風險場景，無法保證說詞的一致性與絕對安全。



LIMITATION

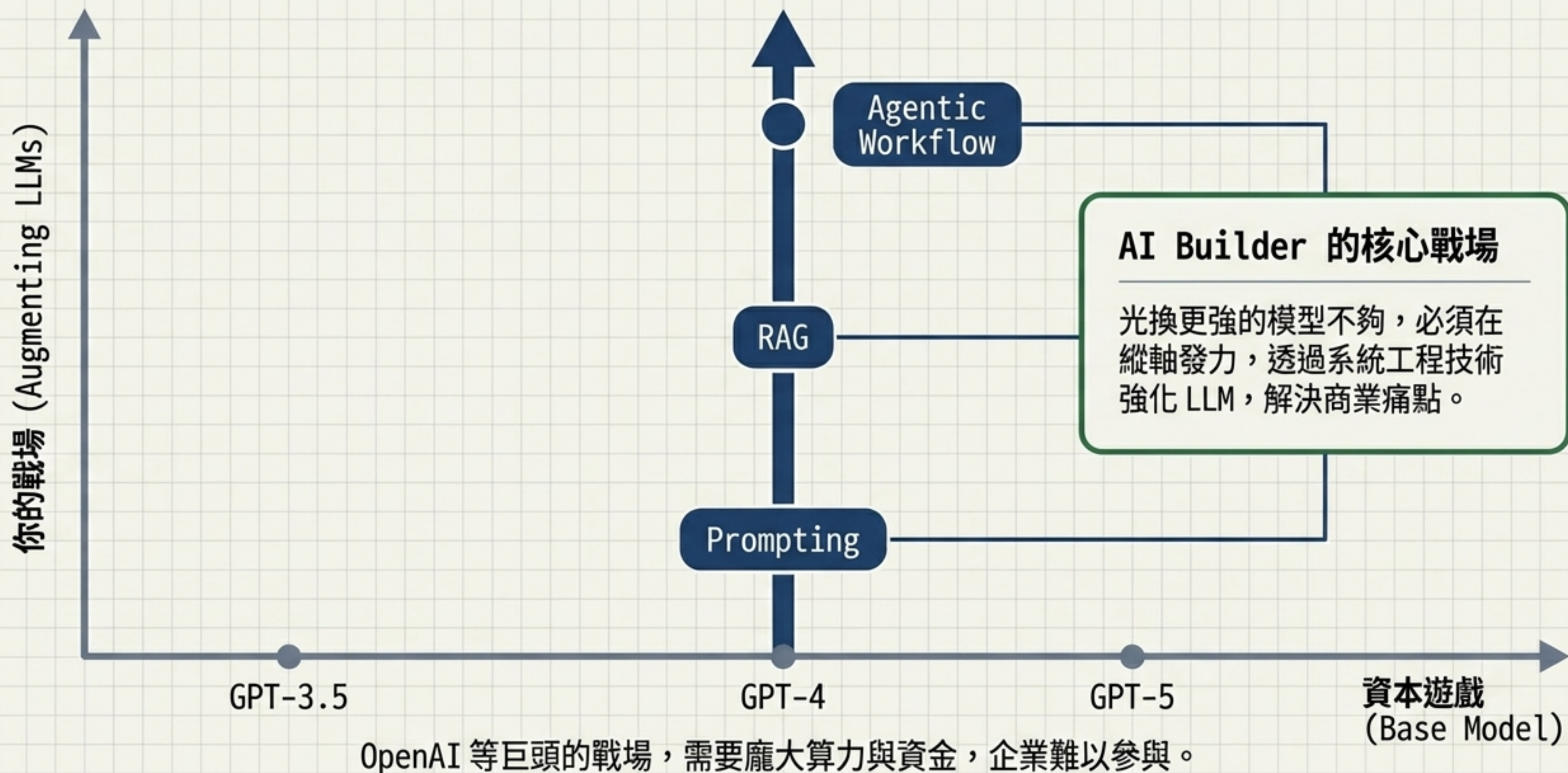
上下文遺失 (Lost in the Middle)

即使 Context Window 達 100 萬 Token，將特定細節（如「Gary 午餐吃蘋果」）塞進海量紀錄中，仍會漏接。



Gary's Apple

AI 開發者的雙軸戰略



Prompt Engineering：從咒語到工程

BCG 人機協作模式 (Collaboration Modes)



半人馬 (Centaur)

分工委派。適合重複性高、流程清楚的任務。
將大任務完整丟給 AI 後，人類執行其他工作。



生化人 (Cyborg)

高頻來回。適合需判斷、創意的任務。
與模型一句一句對話校正，逼出最佳產出。

Prompt Chaining 解構 (Engineering Flow)

傳統黑盒子

單一 Prompt：
讀投訴信並回覆

缺乏可觀測性

工程化拆解

Step 1:
抽取客戶
抱怨意圖

Step 2:
依據意圖
生成大綱

Step 3:
將大綱寫成
完整回信

帶來可觀測性
(Observability)，每一步
皆可獨立測試與 Debug。

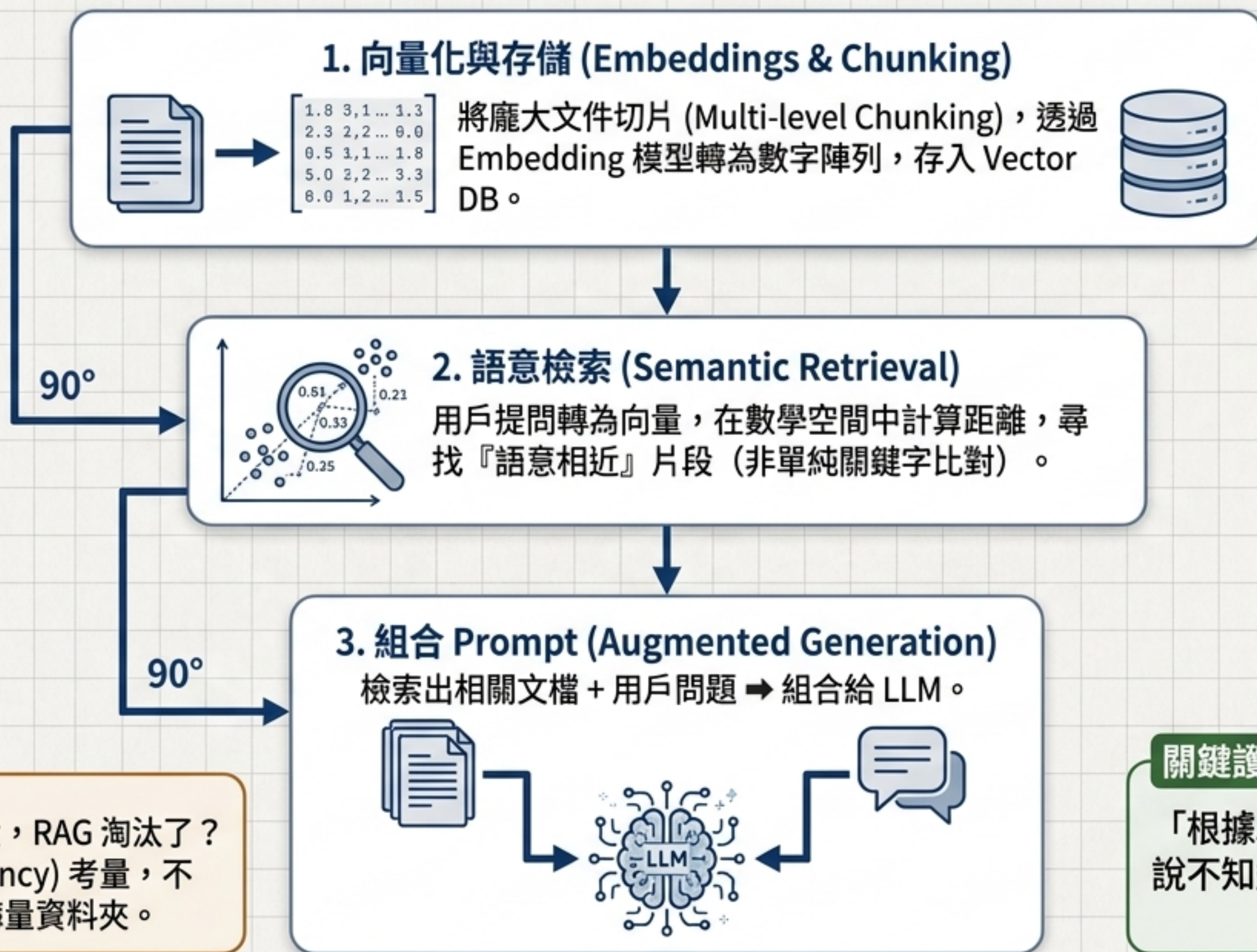
特定領域知識的灌注：Fine-Tuning vs. RAG

	Fine-Tuning	RAG (檢索增強生成)
資料成本	極高 (需大量高品質量身標註資料)	低 (直接利用企業現有文件)
時效性	差 (新 Base Model 釋出即面臨淘汰)	極佳 (可隨時更新外部資料庫)
避免過度擬合 (Overfit)	容易失去通用能力	保留原模型廣度與推理能力
防範幻覺	弱	強 (精準鎖定參考文件範圍)

史丹佛教授結論：

- Fine-Tuning 適用於改變輸出格式或語氣。除此之外，能不做就不做。
- 90% 的企業應用應從 RAG 開始，這是 AI 系統落地的首選。

RAG 運作機制拆解 (Retrieval-Augmented Generation)



迷思破解

Context Window 變大，RAG 淘汰了？
→ 錯！因為延遲 (Latency) 考量，不可能每次提問都重讀海量資料夾。

關鍵護城河指令

「根據以下文件回答。不知道就說不知道。」（防止憑空捏造）

典範轉移：傳統軟體 vs Agentic AI



傳統軟體工程 (Traditional)



A 資料型態: 結構化 JSON / 表單

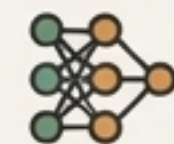
B 邏輯機制: Deterministic
(100% 可預測)

C 架構心態: Microservices
(精確控制每行程式碼)

D 測試方法: 確定性測試
(跑 100 次結果一樣)



AI 系統架構 (Agentic)



A 資料型態: 非結構化自然文本、圖音

B 邏輯機制: Fuzzy
(具機率與隨機性，無單一固定答案)

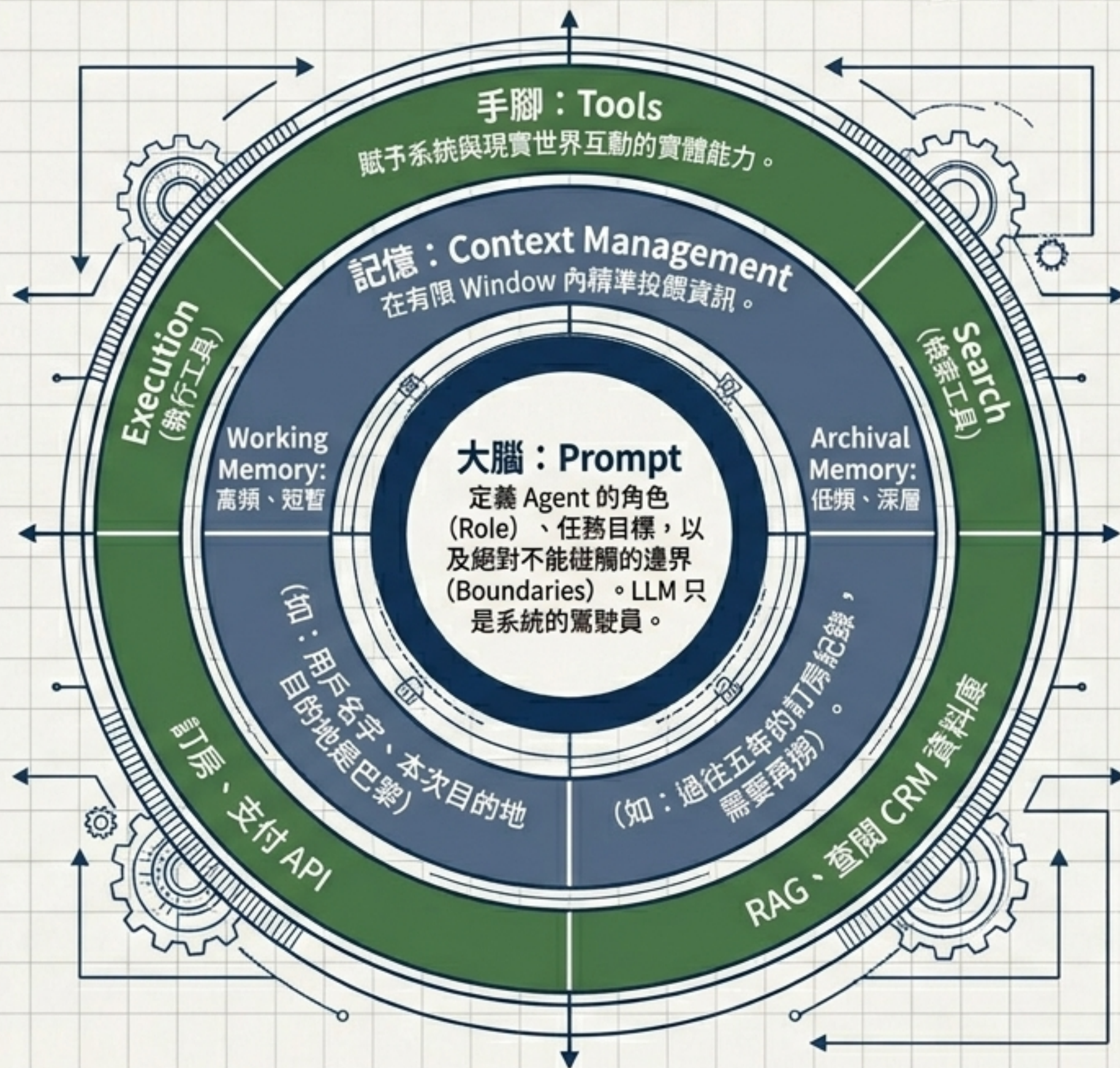
C 架構心態: Think like a Manager
(給予目標與邊界，AI 決定路徑)

D 測試方法: 迭代探索式測試

落地實戰心法 (Golden Rule)

能用 Deterministic 解決的就別用 AI；剩下的 Fuzzy 環節，必須加入「人類申訴 (Human Appeal)」的容錯接手機制。

Agentic Workflow 的三大核心元件

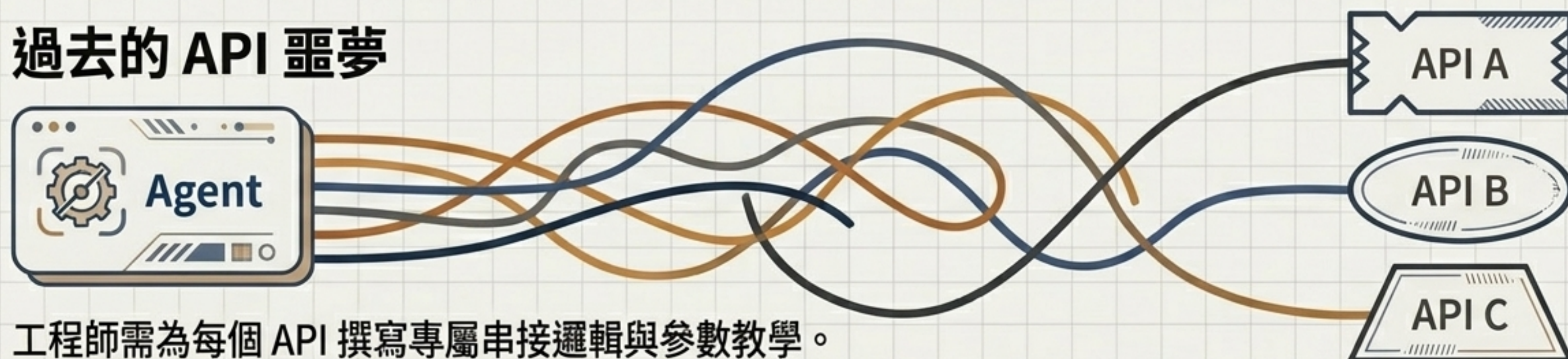


Agent 的自主權光譜 (Levels of Autonomy)



萬能插頭 MCP 協議 (Model Context Protocol)

過去的 API 噩夢



MCP 的破壞性創新



戰略意義：MCP 不只是轉接頭，更是未來 Multi-Agent 彼此點對點溝通的標準語言。

建構 Evals 測試護城河 (評估框架)

沒有評估，系統就無法上線

量化 vs 質化

- Quantitative：修改地址的系統成功率 80%、各環節 Latency 數字。
- Quantitative：各環節一步驟 Latency 數字。
- Qualitative：具體在哪一步產生幻覺？哪一步讓使用者感到困惑？

客觀 vs 主觀

- Objective：訂單 ID 寫入資料庫是否正確？（可全自動化驗證）
- Subjective：客服語氣是否具備同理心？（需依賴人工或 AI 評分）

微觀 vs 宏觀

- End-to-end：使用者整體滿意度打幾分？
- Component-based：單一步驟檢查（如：查地址 API 有無報錯？Email 是否觸發？）

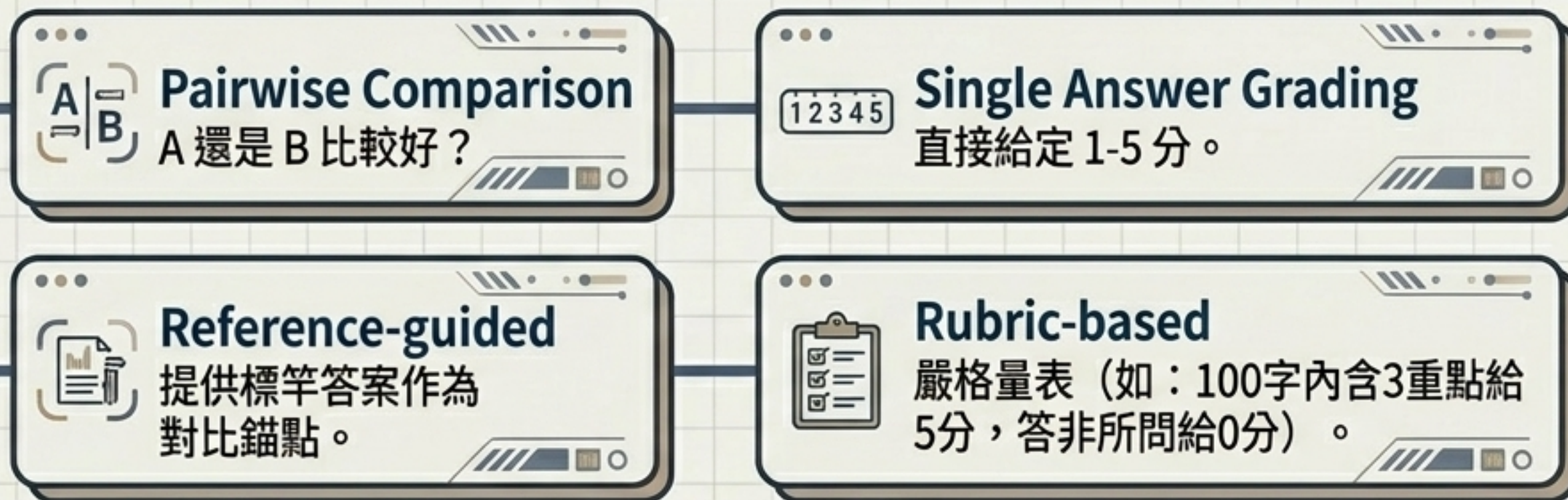
Z-Axis

X-Axis

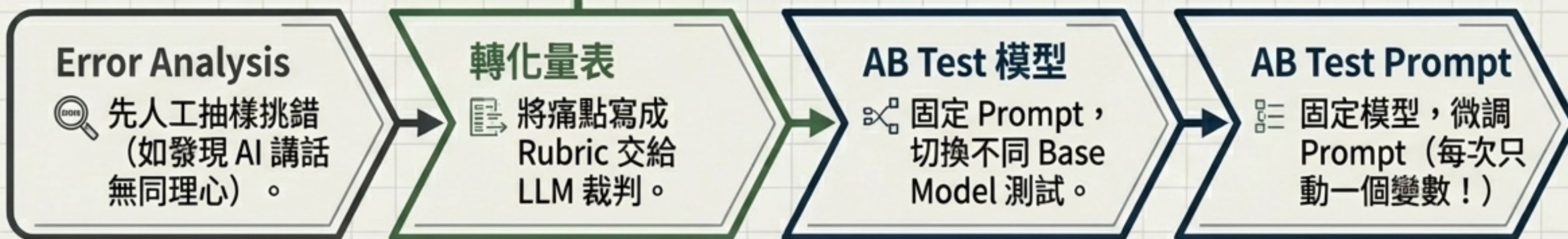
LLM as a Judge 主觀評估法

當沒有標準答案時，如何自動化評分？

四種 AI 裁判模式



實戰落地四步心法



實戰演練：打造 AI 客服 Agent

任務情境：「我要改 A127 訂單的地址，我搬家到建國南路了。」

邏輯步驟

抽取意圖
(Extract ID &
New Address)

查資料庫
(Look up order
history)

查退換政策
(Check if
shipped/address
editable)

起草回信
(Draft response
using gathered
context)

寄出 Email
(Send to
customer)

AI 技術模組

[LLM
(One-shot)]

[MCP /
Custom Tool]

[RAG]

[LLM +
Context Memory]

[Execution
Tool]

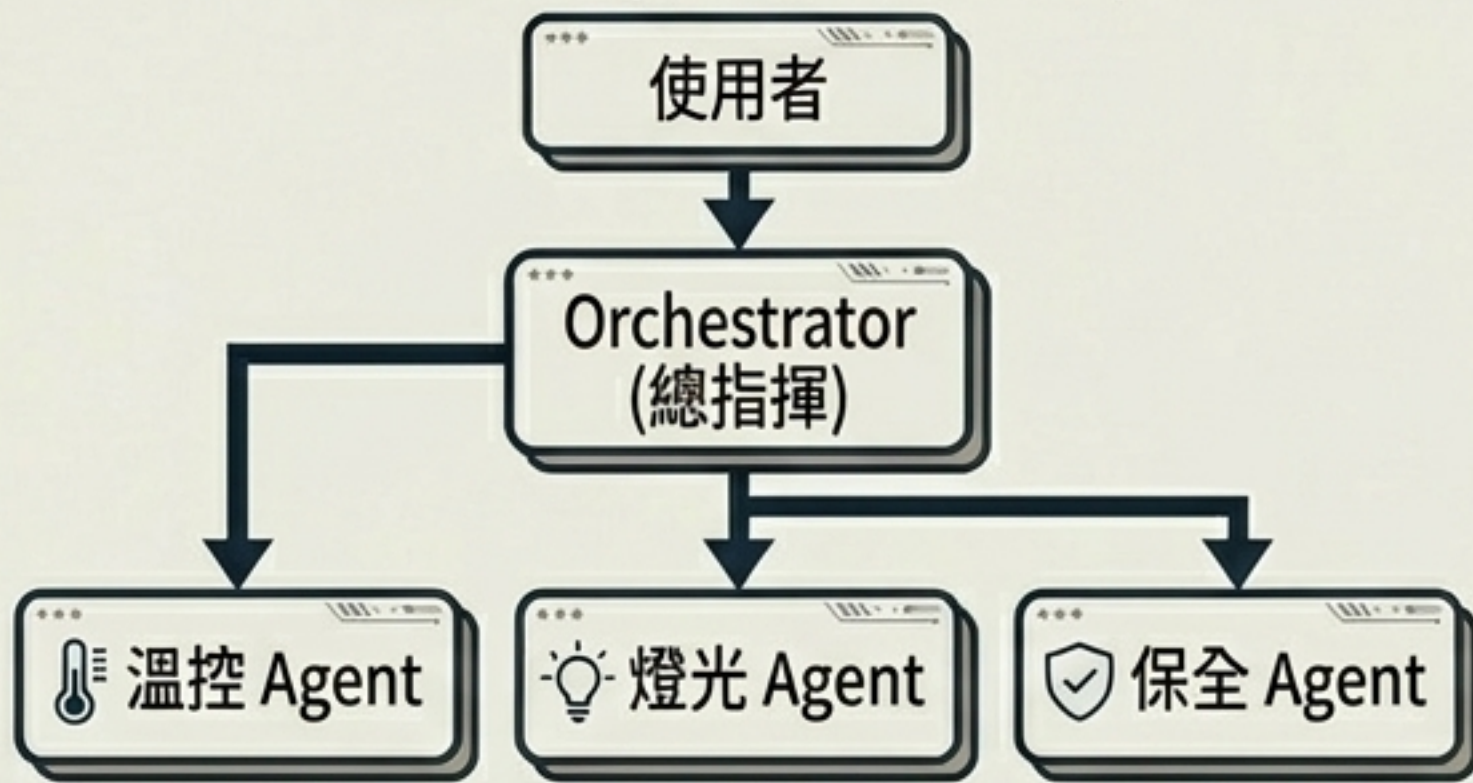
Evals: 抽取準確度

Evals: 語氣與同理心
+ 人類接手機制

Multi-Agent 系統：何時才需要？

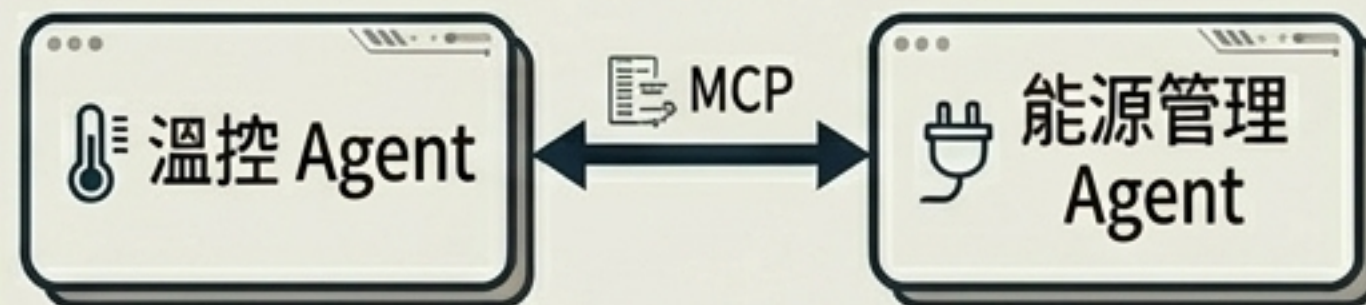
核心價值：解決平行處理 (Parallel Processing) 與模組複用 (Reusability)。

階層式 (Hierarchical)



單一溝通入口，適合智慧家庭。

扁平式 (Flat)



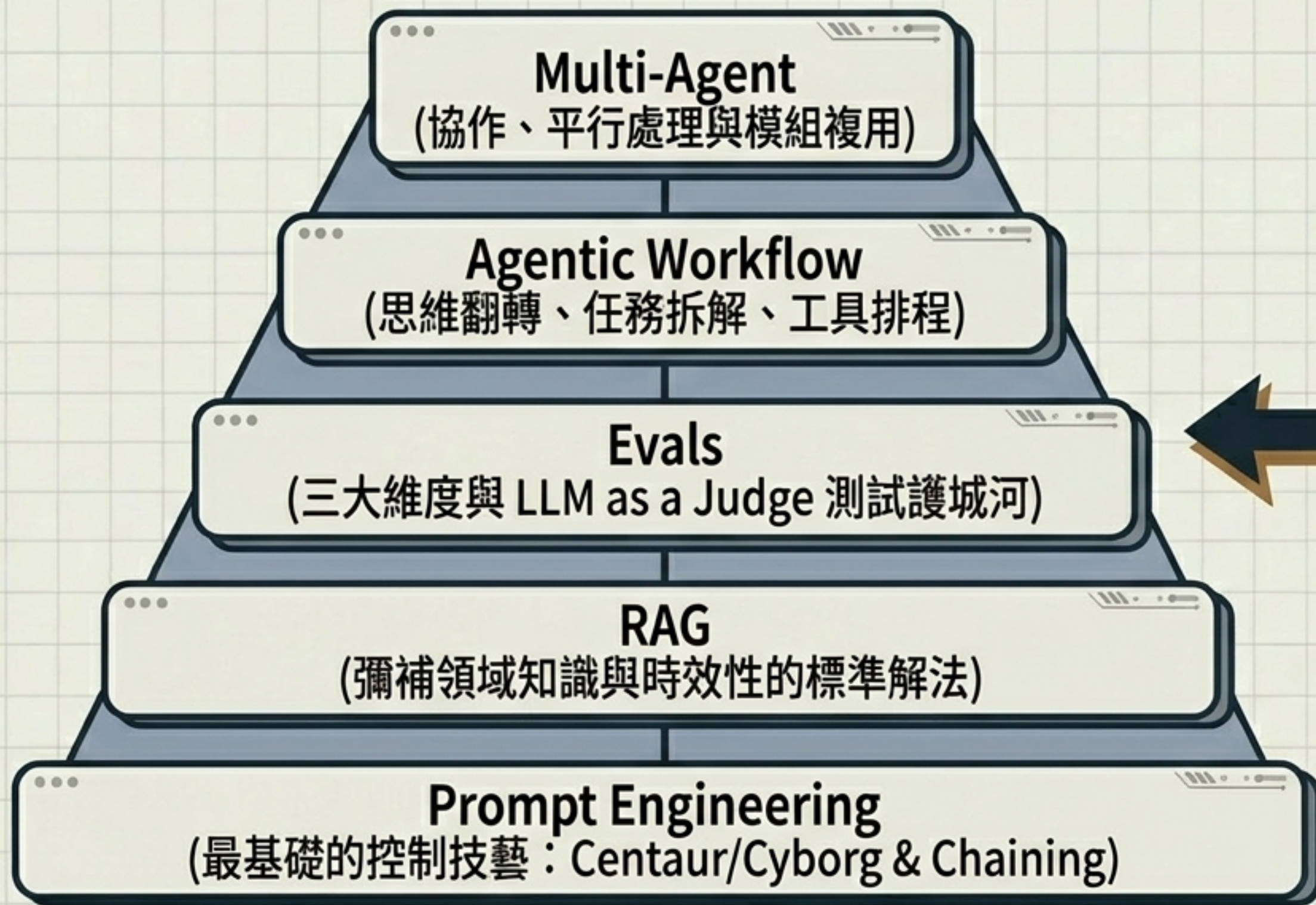
Agent 透過 MCP 直接點對點溝通，
減少總指揮的溝通成本。



架構師防呆提醒

一個 Agent 能解決的，就不要用 Multi-agent。
保持工程設計簡潔，Avoid Overengineering。

AI Builder 的終極藍圖 (The Builder's Roadmap)



行動指南 (Call to Action)

不要盲目追求最炫的技術
(如 Multi-Agent)。
從工作中的「真實痛點」
出發，反向推導你需要爬
到藍圖的哪一個階層。
以實作驅動你的 AI 系統
學習之路。